MANUAL DE INSTALACIÓN Y USO

Plugin transcript-cleanup-toolkit para Claude Code

De audio crudo a transcripción de revisión: Deepgram Nova-3, diagnóstico de calidad de audio, reconstrucción de turnos por hablante y un `.docx` con marcas de tiempo reales, fiel a lo dicho.

github.com/ramonzamora89/transcript-cleanup-toolkit · v0.2.0 · octubre de 2026

Contenido

1 Qué es este plugin (y qué no hace por vos)

2 Requisitos previos

3 Instalación en la terminal (Claude Code CLI)

4 Instalación y uso en la app Claude Desktop (pestaña Code)

5 Montar el proyecto que va a usar el plugin

6 Flujo de trabajo completo, paso a paso

7 Árbol de decisión: cuando Deepgram no devuelve habla

8 Cómo se ha usado en proyectos reales

9 Gotchas ya pagados (tabla de referencia)

10 Qué NO hacer

11 Solución de problemas

12 Referencia rápida

1. Qué es este plugin (y qué no hace por vos)

transcript-cleanup-toolkit es un plugin de Claude Code que empaqueta el flujo completo para convertir una grabación —entrevista, taller, panel, programa de radio o TV— en una transcripción `.docx` de revisión: transcripción con Deepgram Nova-3, diagnóstico de calidad de audio, reconstrucción de turnos por hablante (por *utterance* o por palabra, según cuál capture mejor la conversación) y un documento final con marcas de tiempo reales.

El plugin trae **un skill**, `transcript-cleanup`, con nueve scripts, una página de revisión y dos referencias. Se activa solo dentro de una sesión de Claude Code cuando pedís transcribir o limpiar una grabación; no hay que invocarlo con un comando especial.

Lo que aporta el plugin

- El **pipeline mecánico**: llamada a Deepgram, diagnóstico, construcción de turnos desde el JSON crudo y generación del documento.
- El **árbol de decisión de calidad de audio**: qué medir y en qué orden cuando una grabación devuelve poco o nada.
- Los **principios de limpieza**: no inventar, `[dudoso]` / `[inaudible]` con marca de tiempo exacta, fidelidad sobre pulido.
- La **revisión de dudosos escuchando**: un clip de audio (o video) por cada tramo dudoso, con opciones para elegir o escribir la tuya, en una página local (app Desktop) o en el chat (terminal). Ver §6, Paso 2.5.

Lo que no trae, a propósito

- **Glosario de dominio.** Los nombres propios, siglas e instituciones de cada proyecto viven en el proyecto, no en el plugin.
- **Convención de etiquetas de hablante.** `Hablante 1/2`, `Entrevistador(a)/R` o nombres reales: lo decide cada proyecto.
- **Política de datos sensibles.** Si se tokenizan nombres de personas —y con qué etiquetas— es una decisión del proyecto, no del plugin.

> **LA PARTE QUE SIGUE SIENDO HUMANA**
> La limpieza fina —decidir si un tramo raro es una palabra mal transcrita o un cambio de hablante que la diarización no detectó— no se automatiza. El skill la sistematiza: dice qué comparar, qué marcar y qué dejar registrado. El criterio lo sigue poniendo quien revisa.

2. Requisitos previos

Qué	Para qué	Cómo verificar
Claude Code (CLI o app Desktop)	Ejecutar el skill	`claude --version`
Clave de Deepgram en `DEEPGRAM_API_KEY`	Transcribir	`echo $DEEPGRAM_API_KEY`
ffmpeg	Medir volumen, detectar contrafase, normalizar	`ffmpeg -version`
Python 3	Diagnóstico y construcción de turnos	`python3 --version`
Node.js + librería `docx`	Generar el `.docx` final	`node --version`

La librería `docx` se instala una sola vez dentro del skill:

```
cd <ruta-del-plugin>/skills/transcript-cleanup
npm install
```

> **OJO CON DÓNDE QUEDA NODE_MODULES**
> Si instalás el plugin desde el marketplace, la copia que usa Claude Code vive en `~/.claude/plugins/cache/` y **no** trae `node_modules`. Ver §11 (*«Cannot find module 'docx'»*) para las dos formas de resolverlo.

3. Instalación en la terminal (Claude Code CLI)

El repositorio funciona como su propio *marketplace* de Claude Code: no hace falta que esté en ningún catálogo de Anthropic, basta con que sea un repositorio público con el manifiesto `.claude-plugin/marketplace.json` en la raíz —este ya lo tiene.

Dentro de una sesión de Claude Code, en cualquier proyecto:

```
/plugin marketplace add ramonzamora89/transcript-cleanup-toolkit
/plugin install transcript-cleanup-toolkit@transcript-cleanup-toolkit
```

El nombre después de `@` es el del **marketplace** (definido en `marketplace.json`), que en este caso coincide con el del plugin. Verificá con `/plugin` que aparezca instalado y habilitado.

Qué queda en disco

Qué	Dónde
Registro de marketplaces conocidos	`~/.claude/plugins/known_marketplaces.json`
Clon del marketplace	`~/.claude/plugins/marketplaces/transcript-cleanup-toolkit/`
Copia instalada del plugin	`~/.claude/plugins/cache/transcript-cleanup-toolkit/transcript-cleanup-toolkit/0.2.0/`

Actualizar y desinstalar

```
/plugin marketplace update transcript-cleanup-toolkit   # trae cambios del repo
/plugin                                                  # habilitar, deshabilitar o desinstalar
```

Alternativa sin marketplace (repositorio clonado a mano)

```
claude --plugin-dir /ruta/local/transcript-cleanup-toolkit
```

Carga el plugin solo para esa sesión, sin pasar por GitHub. Útil mientras se edita el propio plugin: los cambios se ven de inmediato, sin reinstalar.

4. Instalación y uso en la app Claude Desktop (pestaña Code)

La app de escritorio de Claude tiene tres pestañas: **Chat** para conversaciones, **Cowork** para trabajo agéntico largo y **Code**, que es Claude Code con interfaz gráfica. Todo lo de este manual corre en la pestaña **Code**.

> **LA CONFUSIÓN MÁS CARA DE ESTA SECCIÓN**
>
> En la barra lateral hay un menú **Customize** → **Plugins**. Ese instalador es para **Chat y Cowork**: su configuración se sincroniza con tu cuenta de claude.ai y **no** alimenta las sesiones de la pestaña Code.
>
> Los plugins de Claude Code —este entre ellos— se instalan **desde dentro de la sesión** de la pestaña Code, y viven en `~/.claude`, compartido con la CLI.

4.1 Instalar el plugin en la pestaña Code

1 Abrir una sesión

Pestaña **Code** → nueva sesión → elegir la **carpeta del proyecto** donde están (o van a estar) los audios. Cada sesión tiene su propia carpeta, su historial y sus cambios.

2 Registrar el marketplace

En el cuadro de texto de la sesión, escribir:

```
/plugin marketplace add ramonzamora89/transcript-cleanup-toolkit
```

Como el registro de marketplaces es el mismo archivo que usa la CLI (`~/.claude/plugins/known_marketplaces.json`), si ya lo agregaste alguna vez desde la terminal, este paso ya está hecho.

3 Instalarlo desde el navegador de plugins

Botón **+** junto al cuadro de texto → **Plugins** → **Add plugin**. Ahí aparece *transcript-cleanup-toolkit* entre los plugins de los marketplaces configurados. **Manage plugins** sirve para habilitarlo, deshabilitarlo o desinstalarlo. El mismo submenú muestra los skills que aporta cada plugin instalado.

4.2 La clave de Deepgram en Desktop (paso obligatorio)

La app de escritorio **no hereda tu entorno de shell completo**. En macOS lee `~/.zshrc` solo para armar el `PATH` y un conjunto fijo de variables de Claude Code; en Windows toma las variables de usuario y sistema pero no lee los perfiles de PowerShell. Es decir: **un `export DEEPGRAM_API_KEY=...` en tu `.zshrc` no llega a la sesión de Desktop.**

Dos formas de resolverlo, en orden de preferencia:

1. **Editor de entorno local.** En el cuadro de texto, abrir el desplegable de entorno, pasar el cursor por **Local** y hacer clic en el ícono de engranaje. Las variables que guardes ahí quedan cifradas en tu máquina y aplican a todas las sesiones locales. Agregá `DEEPGRAM_API_KEY`.
2. **Clave `env` de `~/.claude/settings.json`.** Aplica a las sesiones de Claude Code (no a servidores de desarrollo). Sirve igual para este caso.

> **ALTERNATIVA POR PROYECTO**
>
> Si preferís no guardar la clave a nivel de máquina, dejá un `.env` en la carpeta del proyecto (con `.env` en el `.gitignore`) y cargalo en la terminal integrada antes de correr los scripts: `set -a; source .env; set +a`. Es lo que se hace en el proyecto BID_Cuali.

4.3 La terminal integrada

Ctrl + ` abre un panel de terminal dentro de la sesión, en la misma carpeta de trabajo y con el mismo entorno que ve Claude. Ahí corren cómodo los pasos manuales del flujo: `npm install`, los `ffmpeg` de diagnóstico o `node build_transcript.js`. Solo está disponible en sesiones locales.

4.4 Diferencias con la CLI que importan para este flujo

Tema	CLI	Desktop (pestaña Code)
Plugins	Comando `/plugin`	Botón + → Plugins (interfaz gráfica)
Ajustes y permisos	Los mismos archivos: `~/.claude/settings.json` y el `.claude/settings.json` del proyecto	
Variables de entorno	Las del shell	Editor de entorno local o `env` en settings
Revisión de dudosos (Paso 2.5)	Clips en `clips_dudosos/` + preguntas en el chat de 4 en 4	Página local con reproductor, abierta en el panel del navegador de la app
Varias grabaciones a la vez	Varias terminales	Sesiones en paralelo en la barra lateral
Sesiones en la nube	—	Disponibles, pero el navegador de plugins no funciona ahí: hay que declarar el plugin en `enabledPlugins` del `.claude/settings.json` del repositorio

Para mover una sesión de la terminal a la app: `/desktop`.

5. Montar el proyecto que va a usar el plugin

El plugin es genérico; el proyecto aporta el contexto. Esta es la estructura que quedó probada en los proyectos que ya lo usan:

```
mi-proyecto/
├── .env                          # DEEPGRAM_API_KEY (y en .gitignore)
├── CLAUDE.md                     # convenciones de trabajo del proyecto
├── audio/                        # grabaciones originales
├── deepgram_json/                # respuestas crudas de la API
├── limpieza/                     # carpeta de trabajo del pipeline
│   ├── package.json              # dependencia: docx
│   ├── references/
│   │   ├── glosario-<proyecto>.md        # nombres, siglas, instituciones
│   │   └── limpieza-criterios-<proyecto>.md
│   └── procesados/
│       └── <ID>/                 # una carpeta por grabación
│           ├── merged_words_<ID>.json
│           ├── turns_ts.json
│           ├── cfg.json
│           └── <ID>_transcripcion.docx
└── transcripciones/              # los .docx finales, ya revisados
```

Los dos archivos de referencia del proyecto

El plugin trae `references/criterios-limpieza-ejemplo.md` como plantilla. Copialo al proyecto y completalo. Lo que va en cada uno:

Archivo	Qué documenta
`glosario-<proyecto>.md`	Términos **confirmados** y sus variantes fonéticas en el ASR: nombres de instituciones, siglas, programas, comunidades. Cada entrada dice a qué forma se estandariza y si ya está confirmada (entonces no va como pendiente en el documento).
`limpieza-criterios-<proyecto>.md`	Lo específico de estas grabaciones: si los turnos se construyen por palabra o por utterance y por qué, convención de etiquetas de hablante, política de tokenización, qué se marca `[dudoso]` y qué se corrige de plano.

> **NO MEZCLAR GLOSARIOS ENTRE PROYECTOS**
>
> Es tentador reutilizar el glosario de otro proyecto como base —y está bien copiarlo como plantilla—, pero no como contenido. Si dos proyectos resultan tratar del mismo tema o los mismos actores (pasa más seguido de lo que parece), documentarlo explícitamente en vez de fusionar las configuraciones.

Registrar el flujo en el `CLAUDE.md` del proyecto

Conviene dejar una línea en el `CLAUDE.md` que diga qué skill se usa y con qué configuración, para que cualquier sesión futura no reinvente el flujo. Por ejemplo:

```
| Audio → transcripción | Skill `transcript-cleanup-toolkit:transcript-cleanup`,
  con la configuración de `limpieza/references/` |
```

6. Flujo de trabajo completo, paso a paso

```
audio o video ──(transcribe.sh)──────────────────► <id>_deepgram.json
   ──(diagnose_audio.py)──► ¿problema de audio? → normalizar → re-transcribir
   ──(build_turns.py / build_turns_words.py)──► merged.json / merged_words_<id>.json
   ──(limpieza fina, criterio humano)──────────► turns_ts.json + cfg.json
   ──(find_dudosos.py + make_clips.py)─────────► dudosos.json + clips_dudosos/
   ──(revisión escuchando: página o chat)──────► resoluciones.json
   ──(apply_resoluciones.py)──────────────────► turns_ts.json + pendientes actualizados
   ──(build_transcript.js)────────────────────► <id>_transcripcion.docx
```

#	Paso	Tipo
0	`scripts/transcribe.sh`	Script (llama a Deepgram)
0.5	`scripts/diagnose_audio.py`	Script (verificación obligatoria en audios largos)

1	`scripts/build_turns.py` y `build_turns_words.py`	Script (correr ambos y comparar)
2	Limpieza fina turno por turno	Manual, con Claude asistiendo
2.5	Revisión de dudosos con clips (`find_dudosos.py` , `make_clips.py` , `review_server.py` , `apply_resoluciones.py`)	Scripts + elección humana (opcional)
3	`cfg.json` + `scripts/build_transcript.js`	Manual + script

Paso 0 · Transcribir

```
export DEEPGRAM_API_KEY=...                  # o cargar el .env del proyecto

scripts/transcribe.sh audio/                              # toda la carpeta
scripts/transcribe.sh audio/ transcripciones/             # con carpeta de salida explícita
scripts/transcribe.sh audio/ transcripciones/ "E01.m4a"   # un archivo puntual

KEYTERMS="COVIAL,Guatecompras,CUTRIGUA" scripts/transcribe.sh audio/   # con glosario
```

El script arma un *slug* por archivo y escribe `<salida>/<slug>/<slug>_deepgram.json` . Salta los archivos que ya tienen un JSON válido, así que se puede volver a correr sobre la carpeta completa sin recobrar lo ya transcrito. Detecta el *Content-Type* según la extensión (`.mp3` , `.m4a` / `.mp4` , `.wav`).

Parámetros por defecto: `model=nova-3&detect_language=true&diarize=true&utterances=true&utt_split=0.8&smart_format=true&punctuate=true` .

> **DOS PARÁMETROS QUE CONVIENE REVISAR ANTES DE UNA TANDA LARGA**
>
> `detect_language=true` → `language=es` **.** En grabaciones largas la detección automática descarta tramos enteros de habla real. En un grupo focal del proyecto Xochi costaba cerca del 9 % de las palabras. Si sabés el idioma, fijalo: editá la línea `QS=` de `transcribe.sh` o llamá a `curl` a mano.
>
> `smart_format=true` **corrompe cifras habladas.** «Tres mil seiscientos millones» salió como `600003000` . Si la grabación tiene datos numéricos que importan, ver §7.4.

Keyterms

Términos propios poco comunes: nombres de instituciones, siglas, programas. Máximo ~100 términos cortos; cargar de más empeora los resultados. **No** incluir nombres de personas si el proyecto los va a tokenizar después, ni términos que sesguen lo que se espera oír. Aun con keyterms, el reconocimiento falla: en un programa de 46 minutos con ~35 términos cargados, «Guatecompras» salió como «guarticombras». Revisá variantes fonéticas antes de concluir que un tema no se mencionó.

Paso 0.5 · Diagnóstico

```
python3 scripts/diagnose_audio.py <id>_deepgram.json [--gap-threshold 30] [--wpm-threshold 80]
```

Calcula palabras por minuto y reporta huecos de silencio largos. Una conversación normal ronda **100–150 palabras/min**. Muy por debajo no significa que «no se habló mucho»: casi siempre es un problema de captación. **Correlo siempre en archivos de más de 15–20 minutos, antes de invertir tiempo en limpieza fina.** Si dispara alguna alerta, seguí el árbol de la §7.

Paso 1 · Construir los turnos

```
python3 scripts/build_turns.py       <id>_deepgram.json merged.json
python3 scripts/build_turns_words.py <id>_deepgram.json merged_words_<id>.json
```

Correr **los dos** y comparar el número de turnos:

- `build_turns.py` agrupa por `utterances`, como ya los entrega Deepgram.
- `build_turns_words.py` agrupa palabra por palabra según el `speaker` de cada una.

Si la versión por palabra da **notablemente más turnos**, es porque `utterances` está fundiendo intervenciones cortas de un hablante dentro del turno largo de otro —típico de grabaciones con un solo micrófono o con un hablante dominante—. En el piloto E04S24 del proyecto Xochi pasó de **43 turnos a 75**, y corrigió la mayoría de las atribuciones erróneas sin tocar el contenido. Si ambos métodos dan lo mismo, no hace falta reconstruir nada especial.

> **ETIQUETAS DE HABLANTE**
>
> Si el archivo ya tuvo una limpieza previa con una convención establecida, preservala al reconstruir: no reasignes etiquetas solo porque cambió el orden de primera aparición. Si es la primera pasada, asignar por orden de aparición; si un nombre real se confirma, usarlo.

Paso 2 · Limpieza fina

Es el paso que necesita criterio. Sobre cada turno reconstruido:

1. **No inventar.** Tramo que no se entiende: `[inaudible]`. Probable pero no seguro: `[dudoso: hipótesis]`, con el **timestamp exacto**, no aproximado, para que se pueda ubicar y confirmar contra el audio.
2. **Fidelidad sobre pulido.** Corregir errores del ASR y puntuación; conservar muletillas, regionalismos y disfluencias reales. No «mejorar» el discurso.
3. **Confianza baja ≠ solo error de palabra.** Un tramo raro puede ser un cambio de hablante que la diarización no detectó.
4. **Estandarizar con criterio.** Un término mal transcrito de forma consistente se corrige sin `[dudoso]` *solo si* el contexto interno de la misma grabación lo confirma (se repite, o se dice completo en algún punto).
5. **La confirmación humana manda.** Si quien encargó el trabajo escucha el audio y corrige algo, eso es verdad, no una hipótesis más.
6. **Medir dos veces.** En un lote, procesar y mostrar una grabación completa antes de escalar a las demás.

El resultado se guarda como `turns_ts.json`: una lista de objetos

```
[
  { "i": 0, "label": "Entrevistador (a)", "ts": "00:00:00", "text": "..." },
  { "label": "Nota de revisión", "ts": null, "text": "...", "note": true }
]
```

Los objetos con `"note": true` salen en rojo e itálica en el documento: sirven para dejar constancia de un tramo dudoso sin ensuciar la transcripción.

Paso 2.5 · Revisión de dudosos escuchando los clips

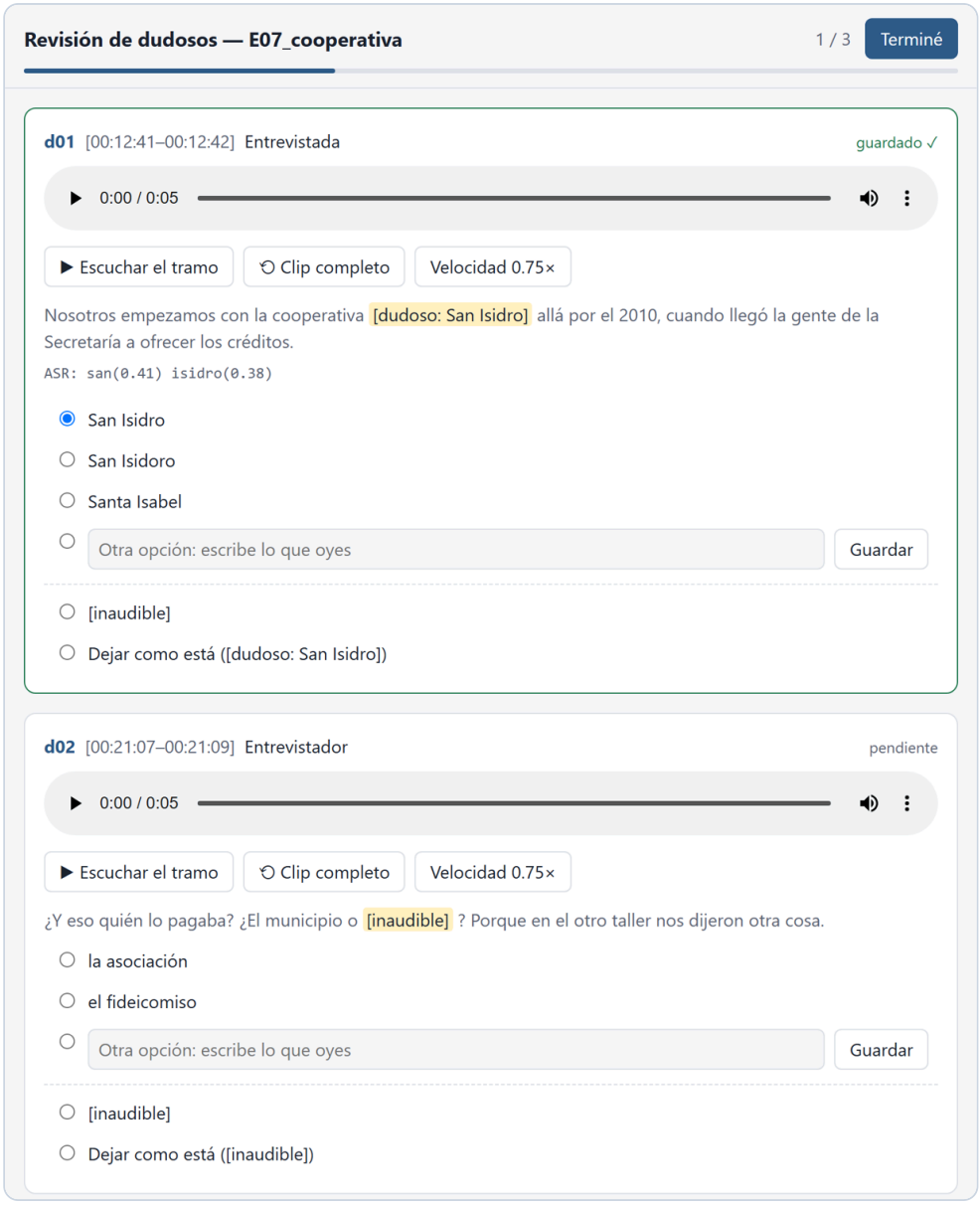
Cómo pedirlo. Al terminar la limpieza fina, Claude ofrece revisar los dudosos. También se puede pedir en cualquier momento: *«revisemos los dudosos de E07 escuchando los clips»*. Si no querés revisar ahora, las marcas quedan como pendientes en el documento, igual que antes.

La limpieza fina deja marcas `[dudoso: …]` e `[inaudible]`. En vez de buscar cada una a mano en el audio, el plugin corta un clip de cada tramo, propone opciones y deja que quien revisa escuche y elija —o escriba lo que oye—. Lo que se elige es confirmación humana: entra al texto sin `[dudoso]`.

```
python3 scripts/find_dudosos.py <dir> <id>_deepgram.json   # ubica cada marca en el tiempo
#   ... Claude redacta 1-3 opciones por tramo en dudosos.json ...
python3 scripts/make_clips.py <dir>                        # clips_dudosos/*.mp3 o *.mp4
python3 scripts/review_server.py <dir>                     # solo en la app Desktop
python3 scripts/apply_resoluciones.py <dir>                # aplica lo elegido
```

- **Ubicación.** `find_dudosos.py` estima `start` / `end` de cada marca con la ventana del turno y las palabras de confianza baja; guarda el texto crudo del ASR para comprobarlo. Si no cuadra, se corrige a mano en `dudosos.json`.
- **Opciones.** Solo con apoyo concreto: el ASR crudo, una variante fonética, el glosario del proyecto o algo dicho en otra parte de la misma grabación. `[inaudible]` y «dejar como está» se ofrecen siempre.
- **Clips.** 1,5 s antes y 1 s después de cada tramo. Si la fuente es video, el clip es `.mp4` con imagen (sirve para ver quién habla); con `--audio-only`, mp3. La ruta de la fuente va en `media` de `cfg.json`.

En la app Desktop (pestaña Code), Claude levanta un servidor local en `http://127.0.0.1:8765/` y abre la página en el panel del navegador de la app: un reproductor por tramo (con «escuchar el tramo», clip completo y velocidad 0,75×), el contexto con la marca resaltada, las opciones y un campo «Otra opción». Cada elección se guarda al momento; el botón «Terminé» cierra el servidor. El audio no sale de la máquina.



La página de revisión en la app Desktop. El primer tramo ya está resuelto; el segundo, pendiente.

En la terminal, Claude te dice dónde quedó `clips_dudosos/` (con un `index.md`) para que abras los clips con tu reproductor, y te pregunta en el chat de **4 en 4** tramos con el selector de opciones; «Otro» es para escribir tu versión.

```
d03 · [00:34:52] d03_003452.mp3
"Las compañeras de ___ siguen yendo cada quince días a la reunión." ¿Qué se dice?

  ❯ 1. Xochitlán
    2. Xochiltepec
    3. [inaudible]
    4. Otro (escribir)
```

Las dos vías escriben el mismo `resoluciones.json`. `apply_resoluciones.py` reemplaza las marcas (respaldo en `turns_ts.before_review.json`) y anota en `pendientes` lo confirmado y lo que quedó sin resolver, con su timestamp.

Paso 3 · `cfg.json` y el documento

```
{
  "id": "E01R19",
  "archivo": "E01R19 Bacajía II San Andrés Villa Seca.m4a",
  "idioma": "Español",
  "duracion": "00:51:42",
  "hablantes": "2 hablantes (diarización automática)",
  "notas": "Contexto real de la grabación, sin saltos de línea.",
  "pendientes": [
    "Turno ~00:49:09: pasaje sobre la línea de quejas quedó con audio poco claro; se dejó [dudoso]."
  ]
}
```

El `id` **controla el nombre del archivo de salida** desde una sola fuente. `notas` no admite `\n`: el generador no convierte saltos de línea. `pendientes` se escribe con lo que de verdad quedó abierto *en esa* grabación —cada `[dudoso]` y cada estandarización con su marca de tiempo—; nunca se hereda una lista fija de otro proyecto. Campos opcionales: `transcripcionBase`, `criterioEdicion` y `estado`, que sobrescriben los textos por defecto de los metadatos.

```
node scripts/build_transcript.js /ruta/al/directorio/de/la/grabacion
```

Lee `cfg.json` + `turns_ts.json` de ese directorio y escribe `<id>_transcripcion.docx` ahí mismo, en Arial, con título, metadatos, notas del archivo, la transcripción (`Hablante N [HH:MM:SS]: texto`) y los pendientes de revisión al final.

> **DECLARAR EL NIVEL DE REVISIÓN**
>
> Un audio de 46 minutos no se revisa turno por turno sin escucha humana. Si se entrega con etiquetas asignadas a mano y glosario aplicado pero sin limpieza fina completa, **decirlo en el propio documento**. El campo `estado` y la sección de pendientes están para eso.

7. Árbol de decisión: cuando Deepgram no devuelve habla

Antes de dar por fallida una grabación, recorrer estos cuatro chequeos en orden. Los cuatro salieron de casos reales.

7.1 ¿Es volumen bajo?

```
ffmpeg -i audio.mp3 -af volumedetect -f null -
```

Un `mean_volume` muy negativo (por debajo de -25 dB; se han visto -29 a -35) lo confirma. Normalizar:

```
ffmpeg -i audio.mp3 -af loudnorm=I=-16:TP=-1.5:LRA=11 -ar 16000 -ac 1 salida.mp3
```

Probar primero en un fragmento de 60–90 s tomado de una zona *sin* habla detectada: mucho más barato que reprocesar el archivo completo y confirma la hipótesis rápido. Si funciona,

normalizar el archivo entero y volver a transcribir, guardando el JSON original como `<id>_deepgram_ORIGINAL_backup.json` antes de sobrescribir. En el proyecto Xochi, normalizar sumado a fijar el idioma recuperó un 13 % de palabras en un mismo archivo.

7.2 ¿Están los canales en contrafase?

Este es el caso que más tiempo puede costar si no se conoce. Un directo de 91,8 minutos devolvió transcripción vacía con confianza 0.0. No era silencio: los dos canales estéreo estaban invertidos en fase. Cada canal medía -33 dB y la suma a mono caía a -58,4 dB —y todo motor de reconocimiento mezcla a mono—.

```
for spec in "c0=c0:L" "c0=c1:R" "c0=0.5*c0+0.5*c1:MID" "c0=0.5*c0-0.5*c1:SIDE"; do
  ffmpeg -i AUDIO -af "pan=mono|${spec%%:*},volumedetect" -f null - 2>&1 | grep mean_volume
done
```

Si MID se hunde frente a L y R, hay contrafase: procesar **un solo canal**, nunca la mezcla.

```
ffmpeg -i AUDIO -af "pan=mono|c0=c0,loudnorm=I=-16:TP=-1.5:LRA=11" -ar 16000 salida.mp3
```

> **CUIDADO CON -V ERROR**
>
> Suprime la salida de `volumedetect` y de `astats`: parece que el comando no hizo nada cuando en realidad sí midió.

7.3 ¿Es eco o reverberación?

Si nova-3 devuelve confianza 0.0 en varios puntos *incluso ya normalizado, con reducción de ruido y exportado a WAV sin comprimir*, el modelo está rechazando el audio: típico de eco fuerte, voces a muy distinta distancia del micrófono o solapamiento constante. Cruzar con Whisper —mismo endpoint de Deepgram, sin cómputo local— sobre un fragmento corto:

```
curl ... "https://api.deepgram.com/v1/listen?model=whisper-
large&language=es&smart_format=true&punctuate=true"
```

Si Whisper saca algo aunque sea parcial, confirma que hay habla real y que el problema es específico de nova-3 con esa acústica.

> **WHISPER NO ES UN REEMPLAZO CONFIABLE**
>
> Alucina texto repetitivo en tramos poco claros. Es un último recurso que necesita revisión humana pesada, no limpieza fina automática: hay que avisarlo explícitamente antes de usarlo sobre un archivo completo, y nunca presentar su salida como transcripción casi lista. Además, el gateway de Deepgram corta los *requests* síncronos largos con `whisper-large`: fragmentar en trozos de ~5 minutos y unir después.

7.4 ¿Hay cifras corrompidas?

`smart_format=true` convierte los números dichos a dígitos y los corrompe: «tres mil seiscientos millones» salió como `600003000`, y «mil trescientos millones» como `300001000` —21 cifras en 92 minutos—. La solución no es adivinar del contexto:

1. Correr el mismo audio con `smart_format=false&numerals=false`.
2. Localizar cada token de seis o más dígitos por su marca de tiempo en la corrida con formateo.
3. Leer la forma dicha en la corrida sin formateo y reconstruir **por alineación de tiempo**.

7.5 Otras dos verificaciones antes de confiar en el resultado

- **La diarización.** Deepgram devolvió *un solo hablante* para 99 segundos de preguntas y respuestas entre una ministra y varios periodistas. Si tanto `build_turns.py` como `build_turns_words.py` dan un turno para un audio que claramente tiene varios hablantes, hay que segmentar a mano sobre las marcas por palabra.
- **Quién habla no lo dice el rótulo del clip.** Un reel rotulado «MINECO destina Q50 millones» mostraba a la viceministra, no a la ministra. Se resolvió extrayendo fotogramas con `ffmpeg` y comparándolos con la fotografía oficial. Mirar el video, no solo escucharlo.
- **Cuando un dato está en disputa, correr el reconocimiento más de una vez.** Tres corridas devolvieron «Valde», «Walde» y «Tribalde» para el mismo apellido. Lo que coincide en dos corridas se puede estandarizar; lo que no, queda `[dudoso]` con su marca de tiempo.

8. Cómo se ha usado en proyectos reales

8.1 Proyecto Xochi (BID_Cuali) — serie de entrevistas de campo

Diagnóstico sobre barreras socioculturales para la vinculación de mujeres a una obra vial en Suchitepéquez y Retalhuleu, Guatemala. Entrevistas semiestructuradas en comunidad, grabadas con un solo micrófono. **13 de las 32 entrevistas** pasaron por el pipeline completo (las 19 anteriores son previas a la herramienta: tokenizadas, pero sin marcas de tiempo confiables ni limpieza fina).

Estructura de trabajo: una carpeta `limpieza-transcripciones-xochi/` con `references/` (glosario + criterios) y `procesados/<ID>/`, una por entrevista, con `merged_words_<ID>.json`, `turns_ts.json`, `cfg.json` y el `.docx`. Los documentos terminados se copian a `TRANSCRIPCIONES/`.

Qué aportó este proyecto al skill:

- La construcción de turnos **por palabra** como opción de primera línea en grabaciones de campo (piloto E04S24: 43 → 75 turnos).
- Fijar `language=es` en vez de `detect_language=true` (+9 % de palabras) y normalizar con `loudnorm` (+13 %) en un mismo grupo focal.
- La regla de que **un «hueco de silencio» del diagnóstico no siempre es silencio**: medir con `volumedetect` y transcribir el fragmento aislado antes de darlo por vacío.
- Tokenización por rol (`[PARTICIPANTE]`, `[ESPOSA]`, `[HIJA N]`) con comunidad y municipio en claro, porque el análisis territorial los necesita. Es política del proyecto, no del plugin.

8.2 Fideicomiso / transporte — declaraciones públicas y cobertura

Registro de declaraciones públicas sobre la respuesta institucional al alza de combustibles. Aquí el material no son entrevistas propias sino clips de radio, TV y redes, más un foro

completo: audio ajeno, sin control sobre la grabación. Estructura: `audio/`, `deepgram_json/`, `docx_build/` y `transcripciones/`.

De este proyecto salieron los chequeos de la §7.2 (contrafase), §7.4 (cifras corrompidas por `smart_format`) y §7.5 (diarización de un solo hablante, identificación por fotogramas, tres corridas para un dato en disputa). También la práctica de **declarar el nivel de revisión** en el propio documento cuando no hubo escucha humana completa.

> **UN DETALLE DE ESTE PROYECTO QUE VALE PARA CUALQUIERA**
> Los reportes que ve un equipo externo no llevan nombres de herramientas, modelos ni costos. Eso vive en el `CLAUDE.md` y el README del proyecto, nunca en el entregable.

8.3 MICIVI — el proyecto que *no* usa este plugin

Análisis de cobertura mediática sobre el Ministerio de Comunicaciones. No tiene audio: trabaja sobre corpus de notas y publicaciones, así que no usa el skill. Se menciona acá por dos razones prácticas: comparte actores y temas con el proyecto de fideicomiso —su archivo de notas sirvió de corpus de contraste— y aporta la regla que allá se confirmó, **clasificar sobre título y entradilla, no sobre el cuerpo**. Cuando un proyecto usa el corpus de otro, se dice explícitamente en el reporte y se aplican *las reglas del proyecto de origen*; las configuraciones no se fusionan.

9. Gotchas ya pagados (tabla de referencia)

Síntoma	Causa real	Qué hacer
Faltan tramos enteros de habla en un audio largo	`detect_language=true` descarta pasajes	Fijar `language=es`
Densidad baja de palabras/min + huecos largos	Volumen de grabación bajo	`volumedetect` → `loudnorm` → re-transcribir
Transcripción vacía, confianza 0.0, audio que sí suena	Canales estéreo en contrafase	Comparar L / R / MID / SIDE; procesar un solo canal
Confianza 0.0 aun normalizado y en WAV	Eco, reverberación o solapamiento	Cruzar con Whisper en un fragmento; revisión humana pesada
Cifras absurdas (`600003000`)	`smart_format` formatea números hablados	Segunda corrida sin formateo; alinear por tiempo
Respuestas cortas atribuidas al hablante equivocado	`utterances` funde intervenciones	Construir turnos por palabra
Un solo hablante en un audio con varios	Diarización fallida	Segmentar a mano sobre las marcas por palabra

Una sigla clave «no aparece» en la transcripción	Variante fonética del ASR	Buscar variantes antes de concluir ausencia
Contenido inservible, todo redactado	`redact=pii`	No usarlo nunca; tokenizar a mano en la limpieza

10. Qué NO hacer

- **No alinear un texto ya «limpio» contra el JSON.** Armar siempre los turnos directo del JSON crudo: alinear rompe las marcas de tiempo y la atribución de hablante en los turnos cortos.
- **No usar `redact=pii` de Deepgram.** Sobre-redacta ocupaciones, fechas, edades y nombres de instituciones, y deja el contenido inservible.
- **No forzar una corrección sin apoyo contextual** solo para «limpiar» un `[dudoso]`.
- **No presentar una salida de Whisper como transcripción lista** para limpieza ligera.
- **No escalar a un lote completo** sin haber validado el criterio en al menos una grabación.
- **No heredar la lista de `pendientes`** de otra grabación ni de otro proyecto.

11. Solución de problemas

`/plugin marketplace add` falla o no encuentra el repositorio

Nombre mal escrito, o el repositorio no tiene `.claude-plugin/marketplace.json` en la raíz. Probar con la URL completa del repositorio.

`/plugin install` dice que no encuentra el plugin

El nombre después de `@` es el del **marketplace**, que puede diferir del nombre del plugin. Acá coinciden: `transcript-cleanup-toolkit@transcript-cleanup-toolkit`.

Instalé el plugin en la app Desktop y el skill no aparece en la pestaña Code

Casi seguro se instaló desde **Customize** → **Plugins**, que alimenta Chat y Cowork, no la pestaña Code. Reinstalarlo desde el botón **+** de la sesión de Code (§4.1).

`ERROR: DEEPGRAM_API_KEY no está definido en el entorno`

En la terminal: `export DEEPGRAM_API_KEY=...` o cargar el `.env`. En la app Desktop: el editor de entorno local o la clave `env` de `~/.claude/settings.json` (§4.2). Recordá que la app **no** lee los `export` de tu `.zshrc`.

`Cannot find module 'docx'`

El generador del documento necesita la librería `docx` de Node, y la copia del plugin en `~/.claude/plugins/cache/` no la trae. Dos salidas:

```
# A) instalarla dentro del propio skill (una vez)
cd ~/.claude/plugins/cache/transcript-cleanup-toolkit/transcript-cleanup-
toolkit/<versión>/skills/transcript-cleanup
npm install

# B) apuntar NODE_PATH a un node_modules que ya la tenga
NODE_PATH=/ruta/a/un/proyecto/node_modules node "$SK/scripts/build_transcript.js" <dir>
```

Si tenés además un *checkout* del repositorio para desarrollo, van a existir dos copias del skill — la del checkout y la del cache—. El contenido es el mismo; lo que cambia es dónde está `node_modules`. Conviene resolver ambas rutas al inicio de la sesión y dejarlo escrito en el `CLAUDE.md` del proyecto.

Git marca todos los archivos como modificados y el diff sale vacío

Pasa cuando el repositorio vive en un disco externo con formato exFAT: git ve un cambio de permisos (`100644 → 100755`) en todo el árbol. `git diff --stat` da 0 inserciones y 0 borrados. Se silencia con `git config core.fileMode false`. No es trabajo sin guardar.

La página de revisión no abre: «el puerto 8765 está ocupado»

Quedó otro servidor corriendo (por ejemplo, una revisión anterior que no se cerró con «Terminé»). Usar `--port 8766`, o cerrar el proceso viejo.

`ERROR: no se encontró ffmpeg` al cortar clips

`make_clips.py` y la extracción de audio de videos necesitan `ffmpeg` y `ffprobe` en el `PATH` de la sesión. En la app Desktop, el `PATH` puede no ser el de tu terminal: agregar la carpeta de ffmpeg en el editor de entorno local (§4.2).

El clip no corresponde al tramo dudoso

La ubicación es una estimación: el texto limpio no se mapea palabra a palabra con el JSON. Corregir `start` / `end` de esa entrada en `dudosos.json` mirando las `words` del JSON de Deepgram, y volver a correr `make_clips.py`.

El `.docx` se ve raro al abrirlo en Word

Abrirlo en Word y exportarlo a PDF antes de darlo por terminado. Word sustituye fuentes y encadena listas numeradas de forma que no se ve en ningún otro visor.

12. Referencia rápida

Comandos

```
# Instalación
/plugin marketplace add ramonzamora89/transcript-cleanup-toolkit
/plugin install transcript-cleanup-toolkit@transcript-cleanup-toolkit
/plugin marketplace update transcript-cleanup-toolkit

# Pipeline
export DEEPGRAM_API_KEY=...
scripts/transcribe.sh audio/ transcripciones/
python3 scripts/diagnose_audio.py transcripciones/<slug>/<slug>_deepgram.json
python3 scripts/build_turns.py       <id>_deepgram.json merged.json
python3 scripts/build_turns_words.py <id>_deepgram.json merged_words_<id>.json
#   ... limpieza fina -> turns_ts.json + cfg.json ...
python3 scripts/find_dudosos.py    transcripciones/<slug> <id>_deepgram.json
python3 scripts/make_clips.py      transcripciones/<slug>
python3 scripts/review_server.py   transcripciones/<slug>   # app Desktop
python3 scripts/apply_resoluciones.py transcripciones/<slug>
node scripts/build_transcript.js transcripciones/<slug>

# Diagnóstico de audio
ffmpeg -i audio.mp3 -af volumedetect -f null -
ffmpeg -i audio.mp3 -af loudnorm=I=-16:TP=-1.5:LRA=11 -ar 16000 -ac 1 salida.mp3
ffmpeg -i audio.mp3 -af "pan=mono|c0=c0,volumedetect" -f null -   # canal izquierdo
```

Contenido del plugin

Archivo	Qué hace
`SKILL.md`	El flujo completo y los principios de limpieza
`scripts/transcribe.sh`	Llama a Deepgram (nova-3, diarización, keyterms)
`scripts/diagnose_audio.py`	Palabras/minuto y huecos de silencio
`scripts/build_turns.py`	Turnos desde `utterances`
`scripts/build_turns_words.py`	Turnos desde el hablante de cada palabra
`scripts/find_dudosos.py`	Ubica en el tiempo cada `[dudoso]` / `[inaudible]`
`scripts/make_clips.py`	Corta un clip por tramo (mp3, o mp4 si la fuente es video)
`scripts/review_server.py` + `assets/review.html`	Página local para escuchar y elegir
`scripts/apply_resoluciones.py`	Aplica las elecciones a `turns_ts.json` y a los pendientes
`scripts/build_transcript.js`	Genera el `.docx` desde `cfg.json` + `turns_ts.json`

`references/criterios-limpieza-ejemplo.md`	Plantilla de glosario y criterios por proyecto
`assets/cfg.example.json`	Plantilla de configuración por grabación

Plugin bajo licencia MIT · github.com/ramonzamora89/transcript-cleanup-toolkit
Este manual se regenera con `python3 docs/build_manual.py` desde `docs/manual.html`.